

```
In [2]: # Import libraries
import pandas as pd
import nltk
import re
import string

from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from sklearn.preprocessing import LabelEncoder
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
In [3]: # Download required resources (run once)
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')

# -----
# Sample Dataset
# -----
data = {
    "text": [
        "I love Machine Learning!!!",
        "NLP is amazing & fun :)",
        "Deep Learning, AI and ML are future.",
        "I hate bugs in code!!!"
    ],
    "label": ["positive", "positive", "positive", "negative"]
}

df = pd.DataFrame(data)

print("Original Data:\n", df)
print("="*60)
```

```
[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\rushi\AppData\Roaming\nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\rushi\AppData\Roaming\nltk_data...
[nltk_data] Unzipping corpora\stopwords.zip.
[nltk_data] Downloading package wordnet to
[nltk_data] C:\Users\rushi\AppData\Roaming\nltk_data...
[nltk_data] Package wordnet is already up-to-date!
[nltk_data] Downloading package omw-1.4 to
[nltk_data] C:\Users\rushi\AppData\Roaming\nltk_data...
```

Original Data:

	text	label
0	I love Machine Learning!!!	positive
1	NLP is amazing & fun :)	positive
2	Deep Learning, AI and ML are future.	positive
3	I hate bugs in code!!!	negative

```
[nltk_data] Package omw-1.4 is already up-to-date!
```

```
In [4]: # -----
# 1. Text Cleaning Function
# -----
def clean_text(text):
    text = text.lower() # lowercase
```

```

text = re.sub(r'\d+', '', text) # remove numbers
text = text.translate(str.maketrans('', '', string.punctuation)) # remove
text = text.strip() # remove extra spaces
return text

df['clean_text'] = df['text'].apply(clean_text)

```

```

In [5]: # -----
# 2. Stopword Removal + Lemmatization
# -----
stop_words = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def preprocess(text):
    tokens = nltk.word_tokenize(text) # tokenize
    tokens = [word for word in tokens if word not in stop_words] # remove stop words
    tokens = [lemmatizer.lemmatize(word) for word in tokens] # lemmatize
    return " ".join(tokens)

df['processed_text'] = df['clean_text'].apply(preprocess)

print("\nProcessed Text:\n", df[['text', 'processed_text']])
print("="*60)

```

```

Processed Text:

```

	text	processed_text
0	I love Machine Learning!!!	love machine learning
1	NLP is amazing & fun :)	nlp amazing fun
2	Deep Learning, AI and ML are future.	deep learning ai ml future
3	I hate bugs in code!!!	hate bug code

```

=====

```

```

In [6]: # -----
# 3. Label Encoding
# -----
label_encoder = LabelEncoder()
df['label_encoded'] = label_encoder.fit_transform(df['label'])

print("\nLabel Encoding:\n", df[['label', 'label_encoded']])
print("="*60)

```

```

Label Encoding:

```

	label	label_encoded
0	positive	1
1	positive	1
2	positive	1
3	negative	0

```

=====

```

```

In [7]: # -----
# 4. TF-IDF Representation
# -----
tfidf_vectorizer = TfidfVectorizer()
tfidf_matrix = tfidf_vectorizer.fit_transform(df['processed_text'])

tfidf_df = pd.DataFrame(
    tfidf_matrix.toarray(),
    columns=tfidf_vectorizer.get_feature_names_out()
)

print("\nTF-IDF Matrix:\n", tfidf_df)

```

TF-IDF Matrix:

	ai	amazing	bug	code	deep	fun	future	hate
\								
0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
1	0.000000	0.57735	0.000000	0.000000	0.000000	0.57735	0.000000	0.000000
2	0.465162	0.000000	0.000000	0.000000	0.465162	0.000000	0.465162	0.000000
3	0.000000	0.000000	0.57735	0.57735	0.000000	0.000000	0.000000	0.57735

	learning	love	machine	ml	nlp
0	0.486934	0.617614	0.617614	0.000000	0.000000
1	0.000000	0.000000	0.000000	0.000000	0.57735
2	0.366739	0.000000	0.000000	0.465162	0.000000
3	0.000000	0.000000	0.000000	0.000000	0.000000

```
In [8]: # -----
# 5. Save Outputs
# -----
# Save full processed dataset
df.to_csv("processed_data.csv", index=False)

# Save TF-IDF features separately
tfidf_df.to_csv("tfidf_features.csv", index=False)

print("\nFiles saved successfully!")
print("1. processed_data.csv")
print("2. tfidf_features.csv")

Files saved successfully!
1. processed_data.csv
2. tfidf_features.csv
```

In []: